MatchACell pipeline

A Snakemake pipeline for single-cell/spatial consensus analysis. Today it implements Step 1 only: a data-driven QC + multi-resolution Leiden clustering + bootstrap-Jaccard cluster-stability optimizer, run once per sample on a raw-ish, post-segmentation `.h5ad`. Additional annotator steps are planned to slot in as further Snakemake rule modules (see "Roadmap" below) — this is a smaller, actively-growing repository, not yet the full multi-step chain its sibling pipelines are.

This document is a reader-facing onboarding + reference guide, generated from `docs/pipeline_guide.md` via `docs/generate_pdf.py`. It complements, but does not replace, the existing hand-written docs in this same folder — `docs/installation.md` (environment setup, CPU/GPU), `docs/methods.md` (the statistical methodology, including CPU-vs-GPU backend divergence), and `docs/cli-and-outputs.md` (the full `matchacell_cluster_stability.py` flag reference and output-file tree) — all still the canonical source for those topics.

Unlike its sibling pipelines in this collection, MatchACell has **no PBS cluster integration** (no `-cl`/`-qu`/`-j` in `run.py`) and **Singularity is off by default** — it ships conda environments, not containers, and is meant to run as one local Snakemake allocation.

Config files at a glance

Per project, this pipeline reads **exactly one YAML file** — no secondary config files at all.

File	Format	Required?	Holds
your project config (`config.yaml`)	YAML	always	output path, sample list, cluster-stability step parameters

---

Quickstart: running your first project

1. Create the conda environment

```
conda env create -f environment.yml      # or: mamba env create -f environment.yml
conda activate matchacell
```

Pins `snakemake-minimal=7.32.4` — needed for `run.py`'s Snakemake Python-API call; the 8.x API is incompatible. GPU acceleration (optional) uses a separate `environment-gpu.yml` template — see `docs/installation.md` before creating it (CUDA-version-specific, must be edited first).

2. Point the config at your data

Copy `config.yaml`, edit `output_dir`, and add `sample_id: path/to/input.h5ad` entries under `samples:`.

3. Dry-run locally

```
./run.py -w cluster_stability -c config.yaml -q 8 -n
```

4. Launch

```
./run.py -w cluster_stability -c config.yaml -q 8
```

No PBS/`-cl` mode exists here — this always runs as one local Snakemake allocation of `-q` cores, with Snakemake itself parallelizing across samples up to that core count.

5. Where outputs land

```
output_dir/results/{sample}/matchacell/
├── clustered_multi_resolution.h5ad     the tracked rule output
└── ...                                  diagnostic plots/tables -- see
docs/cli-and-outputs.md
```

All top-level targets

Target (-w)	What it runs	Requires
`cluster_stability`	QC + multi-resolution Leiden + bootstrap-Jaccard stability, per sample	—

`--list-workflows` prints available targets, including any still-planned ones not yet implemented. Use `--allow-custom-target` to invoke a rule name directly if one is ever added outside `WORKFLOWS`.

Config fields reference

Field	Meaning
`output_dir`	Root output path; a `results/` subfolder is created underneath (trailing slash optional, normalized in the Snakefile)
`extension`	Informational only — documents the expected input file extension (`"h5ad"`); not read to change behavior
`samples`	Map of `sample_ id: path/ to/ input.h5ad` — key is used in output paths and report titles, value is the input file (absolute or relative to the working directory)
`matchacell_ cluster_ stability.backend`	`"auto"` (GPU if `rapids-singlecell` importable, else CPU scanpy), `"gpu"`, or `"cpu"`

`matchacell_ cluster_ stability.n_ iter`	Bootstrap iterations per resolution — higher smooths the Jaccard stability estimate at the cost of runtime
`matchacell_ cluster_ stability.extra`	Extra CLI flags forwarded verbatim to `matchacell_ cluster_ stability.py` (e.g. `"--resolutions 0.2 0.5 1.0 2.0 --skip-tsne"`, `"--gpu-cell-threshold 50000 --rmm-pool-fraction 0.6"`, `"--seed 0 --n-pcs 50 --n-neighbors 15"`) — see `docs/ cli-and-outputs.md` for the complete flag list

Read `--backend gpu` before benchmarking against CPU: the two backends are not numerically equivalent, and GPU is frequently *slower* on small datasets (see `docs/methods.md#cpu-vs-gpu-backend-divergence`).

`run.py` flag reference

```
./run.py -w <target> -c config.yaml -q <cores> [options]
```

G	Flag	Purpose
Re	`-w`/`--workflow`	Target(s) to build
	`-c`/`--configfile`	Path to the YAML config
	`-q`/`--cores`	Cores available to Snakemake
Ex	`-n`/`--dry-run`	Plan jobs without running them
	`-f`/`--forceall`	Rebuild everything
	`-ri`/`--rerun-incomplete`	Rerun jobs with incomplete outputs
	`-u`/`--unlock`	Unlock the working directory
	`--keep-going`	Continue independent jobs after a failure
	`--latency-wait`	Seconds to wait for outputs on slow filesystems
Re	`--list-workflows`	Show targets (including planned ones) and exit
	`-d`/`--dag`	Emit the DAG in DOT format
	`-p`/`--printshellcmds`	Print shell commands
	`-l`/`--lint`	Run Snakemake lint (`text`/`json`)
En	`--no-conda`	Disable per-rule conda (on by default)

	`--conda-prefix`	Shared conda prefix directory
	`--use-singularity`	Enable containers (**off by default** — MatchACell uses conda envs)
	`--bind`, `--singularity-args`	Container bind paths / extra args
	`-di`/`--directory`	Working directory
	`--snakefile`	Custom Snakefile path
	Ad\`--resources`	`key=value` Snakemake resources, e.g. `mem_mb=64000`
	`--allow-custom-target`	Run an arbitrary rule name

`run.py` warns (and, on Snakemake 8.x, fails) if the active Snakemake is not 7.x.

Conda/container environments

- `matchacell` (from `environment.yml`) — the only environment used by the current single rule (`workflow/rules/cluster_stability.smk` declares `conda: "../envs/matchacell.yaml"`); Snakemake builds it per rule when conda integration is enabled (default)
- `matchacell-gpu` (from `environment-gpu.yml`, a **template** — edit `cuda-version` and the RAPIDS pins to match your driver/toolkit before creating it) — optional GPU backend
- No containers/Singularity images are used by default

Module map (`workflow/rules/*.smk`)

- `cluster_stability.smk` — the only module today: QC + multi-resolution Leiden clustering + bootstrap-Jaccard stability, one rule instance per sample, shelling out to `workflow/scripts/matchacell_cluster_stability.py`

Roadmap: adding future annotator steps

The repository is structured so additional steps drop in cleanly:

1. Add a script under `workflow/scripts/` (e.g. `annotate_<tool>.py`).
2. Add a rule module under `workflow/rules/` and `include:` it from the `Snakefile`.
3. Register a new target in `WORKFLOWS` in `run.py` (and remove it from `PLANNED`).
4. Run several annotators **in parallel** by giving Snakemake enough cores and listing multiple targets: `./run.py -w annotate consensus -c config.yaml -q 32`.

Because annotators are independent rules, Snakemake schedules them concurrently up to `--cores`.

Troubleshooting

- **`AttributeError: module 'snakemake' has no attribute 'snakemake'`** — active env is Snakemake 8.x; install `snakemake-minimal=7.32.4` (see `environment.yml`).
- **Singularity errors** — leave Singularity off (the default); MatchACell uses conda envs, not containers.
- **`output_dir` produced a path like `.../outputresults/...`** — already fixed in the Snakefile (the trailing slash is normalized); double-check if you've made custom edits.
- **Conda env path** — the rule references `../envs/matchacell.yaml`; repoint it if you keep a prebuilt environment elsewhere, or run with `--no-conda` inside an already-activated `matchacell` env.

Getting help / more detail

- `docs/installation.md` — CPU/GPU environment setup, example dataset
- `docs/methods.md` — the statistical methodology, including CPU-vs-GPU backend divergence
- `docs/cli-and-outputs.md` — full `matchacell_cluster_stability.py` flag reference + output-file tree

Regenerate this PDF after editing `docs/pipeline_guide.md` via:

```
python docs/generate_pdf.py docs/pipeline_guide.md docs/MatchACell_pipeline_documentation.pdf
```

Needs `pip install --user markdown xhtml2pdf` (plus, on this system's Python 3.8, `Pillow<10` and `reportlab==3.6.13`).